

## Bollmaskin

Madina vet boll, och har därför uppfunnit en bollmaskin för elitbollkunskap för att underhålla alla andra deltagare på IOI som vet boll. Maskinens interna struktur ser ut enligt följande:

- Maskinen består av  $N$  **noder**, indexerade från 0 till  $N - 1$ . Noden  $N - 1$  kallas **roten**.
- För varje nod  $u$  med  $0 \leq u < N - 1$  är **föräldern** till noden  $u$  en nod  $P[u]$  med  $P[u] > u$ , och noden  $u$  är ett **barn** till  $P[u]$ . Roten har ingen förälder. En nod utan barn kallas ett **löv**.

Du känner **inte** till  $N$  eller den föräldern  $P[u]$  för någon nod  $u$ . Istället uppger Madina  $M$ , antalet löv, samt att löven har indexen  $0, 1, \dots, M - 1$ . Din uppgift är att fastställa maskinens interna struktur genom att använda den.

Varje nod i maskinen kan innehålla högst en **elitbollkunskap**, och varje elitbollkunskap har ett **värde** som är ett icke-negativt heltal. Vi säger att en nod har värdet  $x$  om den innehåller en elitbollkunskap med värdet  $x$ . En nod är en **bollvetare** om den innehåller en elitbollkunskap; i annat fall är den en **larpare**. Från början är alla noder larpare.

Du kan utföra två typer av operationer:

- **insert**( $U, X$ ): försöker sätta in ett nytt elitbollkunskap med värde  $X$  vid löv  $U$ .
  - Om löv  $U$  är bollvetare, returnerar operationen `false` utan att sätta in elitbollkunskapen.
  - Om löv  $U$  är larpare placeras elitbollkunskapen vid noden  $U$ . Därefter flyttas elitbollkunskapen upprepade gånger till den nuvarande nodens förälder så länge den föräldern är larpare. Förflyttningen avslutas när elitbollkunskapen når roten eller en nod vars förälder är bollvetare. Operationen returnerar `true`.
- **collect**(): om roten är larpare (vilket innebär att maskinen har noll bollkunskap) returnerar `collect` en tom array. Annars samlar den rekursiva proceduren `traverse`, som beskrivs nedan, värdena för alla elitbollkunskaper som för närvarande befinner sig i maskinen i en array  $S$ . Inledningsvis är arrayen  $S$  tom och `traverse(N - 1)` anropas.

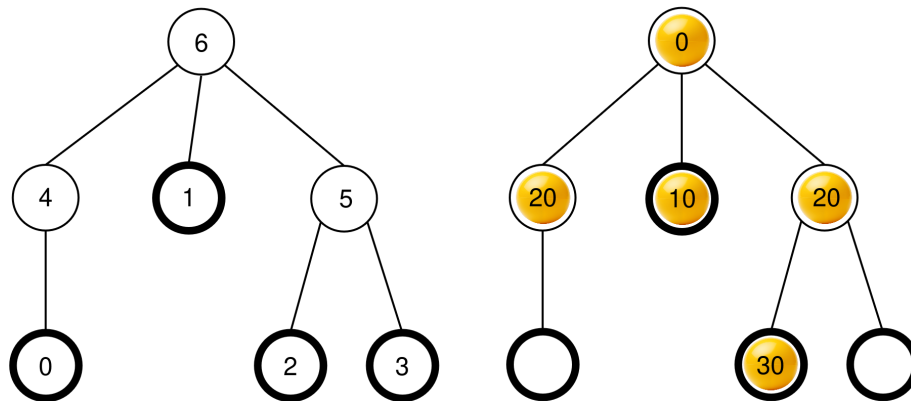
```

traverse(u):
    lägg till värdet på elitbollkunskapen vid noden u i slutet av S
    låt c[u] vara listan över de upptagna barnen till u
    sortera c[u] i icke-avtagande ordning enligt värdena
    för elitbollkunskaperna vid dessa noder (om flera noder har
    samma värde kan de förekomma i valfri ordning)
    för varje v i c[u]:
        traverse(v)

```

När denna procedur är avslutad **tas alla elitbollkunskaper bort** från maskinen, och `collect` returnerar  $S$ .

Tänk dig till exempel en maskin med  $N = 7$  noder och en föreälder-array  $P = [4, 6, 5, 5, 6, 6]$ . Bilden till vänster visar den tomma maskinen med nodindex, medan bilden till höger visar en möjlig konfiguration av elitbollkunskaperna efter en viss sekvens av `insert`-operationer (denna sekvens beskrivs i avsnittet "Exempel").



När funktionen `collect()` anropas är roten (noden 6) bollvetare, så  $S$  initialiseras som  $S = []$  och `traverse(6)` anropas.

**traverse(6):** noden 6 har värdet 0; genom att lägga till den till  $S$  blir resultatet  $S = [0]$ . Barnnoderna till noden 6 är 4, 1 och 5, vilka alla är upptagna. Deras värden är 20, 10 respektive 20. Sortering i icke-avtagande ordning ger  $c[6] = [1, 5, 4]$  (notera att  $c[6] = [1, 4, 5]$  skulle också vara giltigt, eftersom noderna 4 och 5 har samma värde). Funktionen anropar sedan `traverse` på varje nod i  $c[6]$  i den ordningen.

- **traverse(1):** noden 1 har värdet 10; genom att lägga till det till  $S$  får man  $S = [0, 10]$ . Noden 1 har inga upptagna barn, så detta anrop avslutas.
- **traverse(5):** noden 5 har värdet 20; genom att lägga till den till  $S$  blir det  $S = [0, 10, 20]$ . Noden 5 har ett upptaget barn, noden 2, så  $c[5] = [2]$  och proceduren anropar `traverse(2)`.

- **traverse(2)**: noden 2 har värdet 30; genom att lägga till den till  $S$  blir det  $S = [0, 10, 20, 30]$ . Noden 2 har inga upptagna barn, så detta anrop avslutas.
- Exekveringen återgår till **traverse(5)**, som har bearbetat alla barn i  $c[5]$ , så dess anrop avslutas.
- **traverse(4)**: noden 4 har värdet 20; genom att lägga till den till  $S$  blir det  $S = [0, 10, 20, 30, 20]$ . Noden 4 har inga upptagna barn, så detta anrop avslutas.

Alla anrop har slutförts och det finns inga fler noder att bearbeta. Slutligen tas alla elitbollkunskaper bort från maskinen och **collect** returnerar arrayen  $S = [0, 10, 20, 30, 20]$ .

Din uppgift är att bestämma antalet noder  $N$  och rapportera en array med föräldrar  $R = [R[0], R[1], \dots, R[N-2]]$  som beskriver maskinens struktur, men med en eventuellt annorlunda märkning av de noder som varken är bladenoder eller rotnoden. Formellt betraktas en rapporterad array  $R$  som korrekt om det är möjligt att tilldela en unik etikett  $L[u]$  med  $0 \leq L[u] < N$  till varje nod  $u$  i maskinen så att:

- $L[u] = u$  för alla  $0 \leq u < M$  och  $u = N - 1$ , och
- $L[P[u]] = R[L[u]]$  för alla  $0 \leq u < N - 1$ .

Det krävs **inte** att  $R[u] > u$ . Maskinen **måste vara tom** när du lämnar in din lösning.

Låt  $K$  vara antalet utförda **collect**-operationer, och  $B$  vara det största värdet för någon elitbollkunskap över alla **insert**-anrop. Låt  $C = K + B$ . Då får  $C$  **inte överstiga** 1000. Din poäng i vissa deluppgifter beror på värdet av  $C$ .

## Implementationsdetaljer

Du skall bolla med dig själv och implementera följande funktion.

```
std::vector<int> find_structure(int M)
```

- $M$ : antalet blad i trädet.
- Proceduren anropas exakt en gång för varje testfall.
- Proceduren måste returnera en array  $R = [R[0], R[1], \dots, R[N-2]]$  med längden  $N - 1$ , en korrekt array med föräldrar.

För att interagera med maskinen kan din funktion anropa följande två funktioner.

Den första funktionen är:

```
bool insert(int U, int X)
```

- $U$ : indexet för det löv där elitbollkunskapen ska sättas in på. Villkoret  $0 \leq U < M$  måste uppfyllas.
- $X$ : elitbollkunskapens heltalvärde. Villkoret  $0 \leq X \leq 1000$  måste vara uppfyllt.
- Denna funktion returnerar `true` om elitbollkunskapen lyckades, eller `false` om bladet  $U$  redan var upptaget.
- Denna funktion kan anropas högst 500 000 gånger.

Den andra funktionen är:

```
std::vector<int> collect()
```

- Samlar in värdena på alla insatta elitbollkunskapar, tömmer maskinen och returnerar den insamlade arrayen  $S$ .

Om värdet på  $C$  vid något tillfälle under programmets körning överstiger 1000 kommer din lösning att få bedömningen `Output isn't correct: Too many resources used`.

Maskinens struktur fastställs innan funktionen `find_structure` anropas. Gradern är **deterministisk** i den meningen att om man kör den två gånger och båda körningarna utför samma sekvens av operationer, kommer anropen till `collect` att returnera samma arrayer.

## Begränsningar

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

## Deluppgifter

| Deluppgift | Poäng | Ytterligare villkor             |
|------------|-------|---------------------------------|
| 1          | 5     | Roten har exakt $M$ barn.       |
| 2          | 10    | $M \leq 3$                      |
| 3          | 25    | $N \leq 200, M \leq 45$         |
| 4          | 60    | Inga ytterligare begränsningar. |

I deluppgifterna 3 och 4 beror din poäng på värdet på  $C$  enligt följande.

### Subtask 3 (25 points)

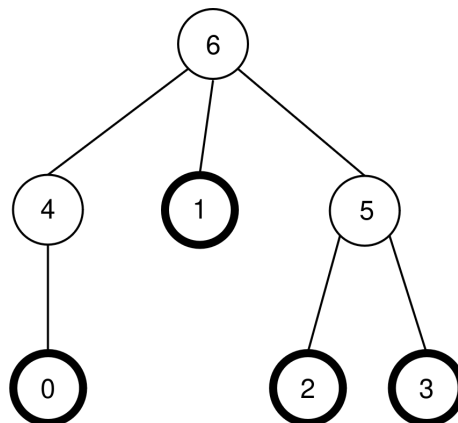
| Begränsningar      | Poäng |
|--------------------|-------|
| $1000 < C$         | 0     |
| $45 < C \leq 1000$ | 13    |
| $C \leq 45$        | 25    |

### Subtask 4 (60 points)

| Begränsningar       | Poäng              |
|---------------------|--------------------|
| $1000 < C$          | 0                  |
| $200 < C \leq 1000$ | 7                  |
| $71 < C \leq 200$   | $47 - \frac{C}{5}$ |
| $44 < C \leq 71$    | $104 - C$          |
| $C \leq 44$         | 60                 |

## Exempel

Låt oss utgå från samma maskin som i uppgiftsbeskrivningen, det vill säga  $N = 7$ ,  $M = 4$  och  $P = [4, 6, 5, 5, 6, 6]$ . Maskinen visas återigen i följande figur:



Gradern utför följande anrop:

```
find_structure(4)
```

Följande kan utföra följande sekvens av anrop:

1. **insert(0, 0):** löv 0 är larpare, så en elitbollkunskap med värdet 0 placeras vid löv 0. Noden  $P[0] = 4$  är larpare, så elitbollkunskapen flyttas till nod 4. Noden  $P[4] = 6$  är larpare,

så elitbollkunskapen flyttas till nod 6. Eftersom nod 6 är roten avslutas förflyttningen. Anropet returnerar `true`.

2. `insert(3, 20)`: löv 3 är larpare, så en elitbollkunskap med värdet 20 placeras vid löv 3. Noden  $P[3] = 5$  är larpare, så elitbollkunskapen flyttas till nod 5. Eftersom  $P[5] = 6$  är bollvetare avbryts förflyttningen. Anropet returnerar `true`.

3. `insert(1, 10)`: löv 1 är larpare, så en elitbollkunskap med värdet 10 placeras vid löv 1. Eftersom  $P[1] = 6$  är bollvetare stannar elitbollkunskapen kvar vid nod 1. Anropet returnerar `true`.

4. `insert(2, 30)`: löv 2 är larpare, så en elitbollkunskap med värdet 30 placeras vid löv 2. Eftersom  $P[2] = 5$  är bollvetare stannar elitbollkunskapen kvar vid nod 2. Anropet returnerar `true`.

5. `insert(1, 25)`: löv 1 är bollvetare, så elitbollkunskapen placeras inte vid löv 1. Anropet returnerar `false`.

6. `insert(0, 20)`: löv 0 är larpare, så en elitbollkunskap med värdet 20 placeras vid löv 0. Nod  $P[0] = 4$  är larpare, så elitbollkunskapen flyttas till nod 4. Eftersom  $P[4] = 6$  är bollvetare avbryts förflyttningen. Anropet returnerar `true`.

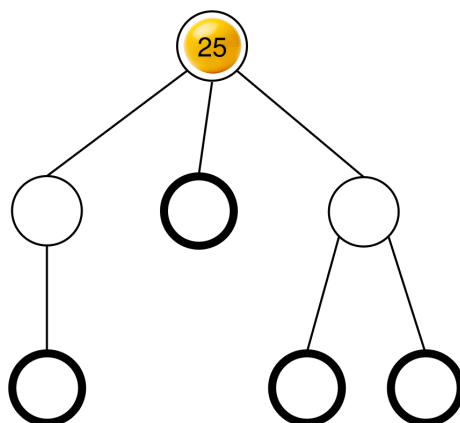
Den slutliga uppställningen av elitbollkunskaparna visades redan i uppgiftsbeskrivningen.

Därefter kan funktionen anropa:

```
collect()
```

Hur detta anrop utförs har redan förklarats i uppgiftsbeskrivningen, så detta anrop returnerar  $S = [0, 10, 20, 30, 20]$ . Därefter är maskinen återigen tom.

Därefter kan proceduren anropa `insert(2,25)`. Löf 2 är larpare, så en elitbollkunskap med värdet 25 placeras vid löf 2. Denna elitbollkunskap rör sig sedan till nod 5 och därefter till nod 6, där den stannar. Anropet returnerar `true`. Den resulterande konfigurationen av elitbollkunskaper visas i följande figur.



Slutligen kan proceduren anropa `collect()`, vilket returnerar  $S = [25]$  och tömmer maskinen.

Det finns två korrekta överordnade arrayer som `find_structure` kan returnera:  
 $R = P = [4, 6, 5, 5, 6, 6]$  (vilket motsvarar märkningen  $L = [0, 1, 2, 3, 4, 5, 6]$ ) och  
 $R = [5, 6, 4, 4, 6, 6]$  (vilket motsvarar märkningen  $L = [0, 1, 2, 3, 5, 4, 6]$ ). I det här exemplet gäller  
att  $K = 2$  och  $B = 30$ , så  $C = 32$ .

## Exempel på exempeldomare

Inputformat:

```
N M  
P[0] P[1] P[2] ... P[N-2]
```

Outputformat:

```
K B C  
Q  
R[0] R[1] R[2] ... R[Q-1]
```

Här är  $Q$  längden på arrayen  $R$  som returneras av `find_structure`.